

**ÇANKAYA UNIVERSITY**  
**FACULTY OF ENGINEERING**  
**COMPUTER ENGINEERING DEPARTMENT**



**CENG 493- Introduction to Natural Language Processing**

**2025-2026 Spring**

**Project**

**Term Project Technical Report**

**Improving Turkish Legal Question Answering with an  
Optimized RAG Pipeline**

**Project Report**

**Names of Members**

**Arda YILDIZ - 202111071**

**Bengisu Yılmaz – 202311086**

**31.05.2026**

## Table of Contents

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>1. Introduction.....</b>                           | <b>3</b>  |
| <b>2. Related Work.....</b>                           | <b>3</b>  |
| 2.1 Retrieval-Augmented Generation.....               | 3         |
| 2.2 Dense Retrieval and Embedding Fine-tuning.....    | 3         |
| 2.3 Hybrid Retrieval.....                             | 4         |
| 2.4 Cross-Encoder Reranking.....                      | 4         |
| 2.5 LLM Fine-tuning for Legal Question Answering..... | 4         |
| 2.6 Turkish NLP and Legal NLP.....                    | 4         |
| 2.7 Evaluation of RAG Systems.....                    | 4         |
| <b>3. Dataset Preparation.....</b>                    | <b>5</b>  |
| 3.1 Collection of Raw Legal Sources.....              | 5         |
| 3.2 Initial Data Inspection.....                      | 5         |
| 3.3 Dataset Format.....                               | 6         |
| 3.4 Data Leakage Prevention.....                      | 9         |
| <b>4. System Architecture.....</b>                    | <b>10</b> |
| 4.1.1 Document Indexing.....                          | 10        |
| 4.1.2 Dense Retrieval.....                            | 10        |
| 4.1.3 Sparse Retrieval (BM25).....                    | 11        |
| 4.1.4 Reciprocal Rank Fusion.....                     | 11        |
| 4.1.5 Cross-Encoder Reranking.....                    | 11        |
| 4.1.6 Prompt Construction.....                        | 11        |
| 4.1.7 Answer Generation.....                          | 11        |
| 4.1.8 Fine-Tuned Models.....                          | 12        |
| <b>5. Evaluation Methodology.....</b>                 | <b>12</b> |
| <b>6. Experiments and Ablation Study.....</b>         | <b>12</b> |
| 6.1 Stage Descriptions.....                           | 12        |
| <b>7. Results.....</b>                                | <b>13</b> |
| 7.1 Base RAG vs Fully Optimized RAG.....              | 13        |
| 7.2 Component Contribution.....                       | 14        |
| <b>10. Conclusion.....</b>                            | <b>14</b> |

# **1. Introduction**

Turkish legal question answering is challenging because legal texts are long, technical, and highly dependent on exact wording and source references. Therefore, answers must be grounded in retrieved legal documents and should minimize hallucinated legal claims. To address this problem, this project uses a Retrieval-Augmented Generation (RAG) approach that combines retrieval and answer generation.

The system takes a Turkish legal question as input, retrieves relevant legal documents, and generates a source-supported answer based on the retrieved context rather than the model's memory. The project compares a baseline RAG pipeline with an optimized pipeline using the same LLM. Components such as embedding fine-tuning, hybrid retrieval, reranking, and LLM fine-tuning are evaluated separately through ablation experiments.

The system also supports custom legal document ingestion and is evaluated in terms of hallucination, grounding, and citation consistency.

## **2. Related Work**

### **2.1 Retrieval-Augmented Generation**

Retrieval-Augmented Generation is a method that combines document retrieval with answer generation. In a standard RAG pipeline, the system first retrieves relevant documents for a user question and then uses these documents as context while generating the final answer. This approach is useful for knowledge-intensive tasks because it reduces the need to rely only on the language model's internal knowledge and helps produce more grounded answers. RAG is especially suitable for legal question answering because legal answers must be supported by reliable sources. In this project, the RAG approach is used to retrieve relevant Turkish legal documents before generating an answer. This helps reduce hallucination and improves the source-supported nature of the generated response.

### **2.2 Dense Retrieval and Embedding Fine-tuning**

Dense retrieval represents both questions and documents as vector embeddings. The system compares the semantic similarity between the question embedding and document embeddings to retrieve relevant passages. This allows the system to find relevant legal documents even when the question and the document do not contain exactly the same words. However, general-purpose embedding models may not fully capture Turkish legal terminology and domain-specific meanings. Therefore, embedding fine-tuning is used to adapt the retriever to Turkish legal question answering. By using query, positive passage, and negative passage examples, the embedding model can learn to retrieve more relevant legal contexts.

## **2.3 Hybrid Retrieval**

Hybrid retrieval combines semantic retrieval with keyword-based retrieval methods such as BM25. Dense retrieval is useful for semantic similarity, while BM25 is useful for exact keyword matching. This combination is important in legal question answering because legal texts often include exact law names, article numbers, procedural terms, and specific legal expressions. In this project, hybrid retrieval is used to improve the retrieval stage by combining dense search and lexical search. This allows the system to benefit from both semantic meaning and exact legal term matching.

## **2.4 Cross-Encoder Reranking**

The first retrieval step may return relevant documents, but they may not always be ordered correctly. A reranker is used to reorder the retrieved candidate passages according to their relevance to the user question. Cross-encoder rerankers process the query and passage together, which allows them to better understand the relationship between the question and the document.

Reranking is important for legal RAG because the LLM should receive the most relevant legal passages as context. If the retrieved context is more accurate, the generated answer is more likely to be grounded, correct, and citation-consistent.

## **2.5 LLM Fine-tuning for Legal Question Answering**

Large Language Models can be adapted to a specific task or domain through instruction tuning and parameter-efficient fine-tuning methods such as LoRA or QLoRA. In legal question answering, fine-tuning can help the model produce more structured, context-aware, and citation-supported answers. However, LLM fine-tuning alone is not enough for legal QA. The answer must still be based on retrieved legal documents. Therefore, this project combines retrieval-aware prompting and LLM fine-tuning within a RAG pipeline.

## **2.6 Turkish NLP and Legal NLP**

Turkish legal question answering is challenging because Turkish is morphologically rich and legal texts contain long sentences, formal expressions, domain-specific terminology, and references to laws or article numbers. Many retrieval and generation models are mainly developed for English, which can limit their performance in Turkish. Legal NLP also requires high accuracy because unsupported or incorrect answers may be risky in legal contexts. For this reason, Turkish legal RAG systems require careful data preparation, retrieval optimization, reranking, and hallucination analysis.

## **2.7 Evaluation of RAG Systems**

RAG systems should be evaluated from multiple perspectives. Retrieval quality can be measured with metrics such as Recall@5, Recall@10, MRR, and nDCG. Answer quality can be evaluated using metrics such as Exact Match, F1, ROUGE, or BERTScore. In addition, grounding quality should be measured with faithfulness, citation accuracy, and hallucination analysis. This project follows a similar evaluation strategy by measuring retrieval performance, answer quality, grounding, and citation consistency. The ablation study also helps show the separate contribution of embedding fine-tuning, hybrid retrieval, reranking, and LLM fine-tuning.

### **3. Dataset Preparation**

Dataset preparation was carried out as a multi-step process because the project uses a modular Retrieval-Augmented Generation pipeline. The system includes retrieval, reranking, answer generation, and evaluation components. Therefore, a single dataset format was not sufficient. Instead, separate datasets were prepared for each component: retrieval corpus, embedding fine-tuning, reranker fine-tuning, LLM supervised fine-tuning, and evaluation.

#### **3.1 Collection of Raw Legal Sources**

The first step was to collect publicly available Turkish legal datasets from different sources. These sources included legal question-answer datasets, Turkish legal chatbot data, Constitutional Court decision summaries, Yargitay retrieval datasets, and large-scale Turkish legal document collections. The main raw sources used in the project were:

- [https://huggingface.co/datasets/OrionCAF/turkish\\_law\\_qa\\_dataset](https://huggingface.co/datasets/OrionCAF/turkish_law_qa_dataset)
- <https://www.kaggle.com/datasets/batuhankalem/turkishlaw-dataset-for-llm-finetuning>
- <https://huggingface.co/datasets/Renicames/turkish-lawchatbot>
- <https://huggingface.co/datasets/KocLab-Bilkent/turkish-constitutional-court>
- [https://github.com/koc-lab/yargitay\\_retrieval\\_dataset](https://github.com/koc-lab/yargitay_retrieval_dataset)
- [https://huggingface.co/datasets/samet713/9\\_yargitay\\_kararlari\\_2025](https://huggingface.co/datasets/samet713/9_yargitay_kararlari_2025)
- <https://huggingface.co/datasets/newmindai/caselaw-retrieval>
- <https://huggingface.co/datasets/erdem-erdem/Turkish-Law-Documents-700k-clustered>

#### **3.2 Initial Data Inspection**

After collecting the raw data, each dataset was inspected before being used in the RAG pipeline. The raw sources were not inserted directly into the system because they contained several data quality problems, such as duplicated records, incomplete or truncated passages, noisy text fragments, inconsistent formatting, broken encoding, overly short entries, source ambiguity, repeated

question-answer pairs, and content that could create data leakage if used without filtering. This inspection step was necessary because legal RAG systems require reliable, clean, and traceable source text. In the legal domain, generating a fluent answer is not sufficient; the answer must also be grounded in a retrieved legal source and supported with consistent citation metadata.

### 3.3 Dataset Format

#### **corpus.jsonl — 7,643 legal chunks**

The format is:

```
{
  "id": "...",
  "text": "...",
  "title": "...",
  "metadata": {
    "source": "...",
    "source_file": "...",
    "doc_id": "...",
    "chunk_id": "...",
    "chunk_index": 0,
    "category": "...",
    "semantic_topic": "...",
    "citation_label": "...",
    "verification_required": false,
    "source_location": "...",
    "language": "tr",
    "previous_chunk_id": "...",
    "next_chunk_id": "..."
  }
}
```

The corpus is the main knowledge base of the RAG system. It does not contain ready-made question-answer pairs. This is important because the model should retrieve relevant legal context and then generate an answer based on that context. If direct question-answer pairs were placed into the corpus, the system could retrieve memorized answers instead of performing grounded retrieval. Each chunk includes citation metadata such as source, source file, document id, chunk id, citation label, and source location. This allows each generated answer to be connected back to the retrieved legal source.

#### **embedding.jsonl — 1,949 triplet examples**

The format is:

```
{
  "id": "...",
  "query": "...",
  "positive_passage": "...",
  "negative_passage": "...",
  "positive_id": "...",
  "negative_id": "..."
}
```

```

    "positive_citation": "...",
    "negative_citation": "...",
    "negative_type": "...",
    "source": "...",
    "metadata": {
        "positive_source_file": "...",
        "negative_source_file": "...",
        "positive_category": "...",
        "negative_category": "..."
    }
}

```

Each example contains a query, a relevant positive passage, and an irrelevant or less relevant negative passage. The purpose is to teach the embedding model to place the query and the correct legal passage closer in vector space while moving irrelevant passages farther away. This dataset improves retrieval quality, but it does not teach the model to generate answers.

The reranker dataset was created to improve the ordering of retrieved documents. In a RAG system, the retriever may find relevant documents, but the most relevant document may not always be ranked first. Therefore, a reranker is used after the initial retrieval stage. The final file is:

### **reranker.jsonl — 6,357 query-passage pairs**

The format is:

```

{
    "id": "...",
    "query_id": "...",
    "query": "...",
    "candidate_passage": "...",
    "label": 1,
    "candidate_id": "...",
    "citation_label": "...",
    "source": "...",
    "negative_type": "...",
    "audit_status": "..."
}

```

The reranker learns whether a query-passage pair is relevant or irrelevant. This is especially useful in the legal domain because exact wording, legal concepts, article numbers, and source context can affect relevance.

### **llm.jsonl — 13,504 examples**

The format is:

```

{
    "id": "...",
    "messages": [
        {
            "role": "system",
            "content": "..."
        },
    ],
}

```

```

{
    "role": "user",
    "content": "..."
},
{
    "role": "assistant",
    "content": "..."
}
],
"metadata": {
    "source_id": "...",
    "corpus_row_id": "...",
    "citation_label": "...",
    "source": "...",
    "source_file": "...",
    "chunk_id": "...",
    "title": "...",
    "variant_type": "...",
    "generated_from": "...",
    "generation_method": "...",
    "requires_manual_review": false
}
}

```

Each example includes a system instruction, a user message containing legal context and a question, and an assistant answer grounded in the given source. The purpose is not to make the model memorize legal information. Instead, the purpose is to teach source-grounded answer generation, legal answer style, and citation-aware response behavior.

Two evaluation datasets were prepared: `rag_eval.json` and `gold_benchmark.json`.

### **`rag_eval.json` — 1,000 evaluation queries**

```

{
    "query_id": "...",
    "query": "...",
    "gold_chunk_ids": [],
    "gold_citation_labels": [],
    "source": "...",
    "source_file": "...",
    "title": "...",
    "category": "...",
    "semantic_topic": "...";
    "gold_answer_extract": "...",
    "metadata": {
        "row_id": "...",
        "verification_required": false,
        "low_quality_text": false,
        "index_only": false,
        "legal_answer_source": true,
        "audit_status": "..."
    }
}

```



```
}
```

This dataset is used to measure retrieval performance with metrics such as Recall@K, MRR, source coverage, and citation correctness.

#### **gold\_benchmark.json — 240 review-ready legal questions**

```
{
  "question_id": "...",
  "question": "...",
  "verified_answer": "...",
  "gold_sources": [],
  "answer_type": "...",
  "difficulty": "...",
  "benchmark_status": "...",
  "manual_legal_review_recommended": true,
  "validation_flags": [],
  "quality_profile": {},
  "notes": "..."
}
```

This dataset is used for final answer quality evaluation. It contains verified answers and gold sources. It is used to evaluate answer correctness, faithfulness, and citation consistency.

The final dataset stack used in this project is:

| Dataset           | File                | Size            | Purpose                            |
|-------------------|---------------------|-----------------|------------------------------------|
| Retrieval Corpus  | corpus.jsonl        | 7,643 chunks    | Main legal source base for RAG     |
| Embedding Dataset | embedding.jsonl     | 1,949 triplets  | Embedding fine-tuning              |
| Reranker Dataset  | reranker.jsonl      | 6,357 pairs     | Cross-encoder reranker fine-tuning |
| LLM SFT Dataset   | llm.jsonl           | 13,504 examples | Grounded QA                        |
| Gold Benchmark    | gold_benchmark.json | 240 questions   | Final QA and citation evaluation   |
| RAG Eval Set      | rag_eval.json       | 1,000 queries   | Retrieval and pipeline evaluation  |

### **3.4 Data Leakage Prevention**

Data leakage was carefully checked during dataset preparation, since the corpus, training datasets, and evaluation datasets were all generated from the same cleaned legal source pool. To prevent the model from seeing evaluation content during training, separate no-leak versions of the embedding, reranker, and LLM fine-tuning datasets were prepared. In this process, all source chunks used in **rag\_eval.json** and **gold\_benchmark.json** were removed from the training datasets. The final leakage control compared evaluation and benchmark source IDs with embedding positive/negative IDs, reranker candidate IDs, and LLM metadata source IDs. Exact query overlaps and conflicting reranker labels were also checked. No ID-level overlap was found between the final training datasets

and the evaluation or gold benchmark source IDs. Therefore, the final dataset stack was considered safe from direct evaluation leakage.

## 4. System Architecture

The Turkish legal RAG system is implemented as a modular pipeline that processes a natural-language legal question through four sequential stages: retrieval, optional reranking, prompt construction, and answer generation. The eight pipeline configurations evaluated in this project are incremental variants of this core architecture, each activating or substituting one component to enable controlled ablation analysis.

| Stage       | Embedding   | Retrieval                      | Reranker | LLM           |
|-------------|-------------|--------------------------------|----------|---------------|
| S1 Baseline | Base BGE-M3 | Dense only (FAISS)             | —        | Base Qwen2.5  |
| S2 EmbedFT  | FT BGE-M3   | Dense only (FAISS)             | —        | Base Qwen2.5  |
| S3 Sparse   | —           | BM25 only                      | —        | Base Qwen2.5  |
| S4 Hybrid   | FT BGE-M3   | Dense + BM25 $\rightarrow$ RRF | —        | Base Qwen2.5  |
| S5 BaseRR   | FT BGE-M3   | Dense + BM25 $\rightarrow$ RRF | Base CE  | Base Qwen2.5  |
| S6 FTRR     | FT BGE-M3   | Dense + BM25 $\rightarrow$ RRF | FT CE    | Base Qwen2.5  |
| S7 LLMFT    | FT BGE-M3   | Dense + BM25 $\rightarrow$ RRF | —        | QLoRA Qwen2.5 |
| S8 Full     | FT BGE-M3   | Dense + BM25 $\rightarrow$ RRF | FT CE    | QLoRA Qwen2.5 |

### 4.1.1 Document Indexing

At index construction time, all corpus chunks are encoded using the selected embedding model (base or fine-tuned BGE-M3) with L2-normalized embeddings. The resulting vectors are stored in a FAISS IndexFlatIP index, which performs exact inner-product search equivalent to cosine similarity for normalized vectors. The FAISS index and document list are cached to disk and reloaded if the corpus size matches, avoiding redundant re-encoding across runs. The embedding dimension is 1024 for BGE-M3.

### 4.1.2 Dense Retrieval

At query time, the question is encoded by the query encoder (same model used for indexing) and searched against the FAISS index to retrieve the top-K candidates. The configuration uses `TOP_K_DENSE = 20` for the dense pass. Each retrieved candidate is returned with its document fields and the associated dense similarity score.

### 4.1.3 Sparse Retrieval (BM25)

In hybrid stages (S3, S4, S5, S6, S7, S8), a BM25Okapi index is built from the tokenized corpus using `rank_bm25`. Tokenization applies lowercase normalization and punctuation removal, retaining only tokens of length greater than one. At query time, the same tokenization is applied to the question and BM25 scores are computed for all corpus chunks. The top `TOP_K_SPARSE = 20` documents are returned with their BM25 scores. Stage 3 uses BM25-only retrieval as an isolated sparse baseline.

### 4.1.4 Reciprocal Rank Fusion

In hybrid stages, dense and sparse candidate lists are merged using Reciprocal Rank Fusion (RRF). For each candidate, the RRF score is computed as:  $\text{score}(d) = 1/(k + \text{rank\_dense}) + 1/(k + \text{rank\_sparse})$ , where  $k = 60$  is the smoothing constant. Documents appearing in only one list receive a contribution from that list only. The merged list is sorted by descending RRF score, producing a unified ranked pool for reranking or generation.

### 4.1.5 Cross-Encoder Reranking

In Stages S5 and S6, the top `TOP_K_RERANK_IN = 20` candidates from the RRF fusion step are passed to a cross-encoder reranker. The reranker processes each (query, passage) pair jointly and produces a scalar relevance score. Stage 5 uses the base model cross-encoder/ms-marco-MiniLM-L-12-v2. Stage 6 uses a fine-tuned version trained on Turkish legal query-passage pairs. The top `TOP_K_FINAL = 5` passages after reranking are passed to the language model.

### 4.1.6 Prompt Construction

The prompt is constructed by concatenating the top-5 retrieved passages into a structured context block. Each passage includes its document index, title, and citation label. The system prompt instructs the model to answer strictly from the provided context and include a citation at the end of the response. The user prompt contains the legal question and requests a concise cited answer.

### 4.1.7 Answer Generation

In all stages, answer generation uses Qwen/Qwen2.5-7B-Instruct loaded in 4-bit NF4 quantization via `BitsAndBytesConfig` (double quantization, `bfloat16` compute). Generation uses greedy decoding (`do_sample=False`) with `repetition_penalty = 1.1`. In Stages 1-6 the base model is used directly. In Stages 7 and 8 a QLoRA fine-tuned adapter is loaded via PEFT on top of the base model. `MAX_NEW_TOKENS` is 128 for Stages 1-4 and 7, and 256 for Stages 5, 6, and 8.

#### 4.1.8 Fine-Tuned Models

Three models were domain-adapted for this project. The BGE-M3 embedding model was fine-tuned using contrastive learning on Turkish legal query-passage pairs and stored at the `bgem3/` path in the model directory. The cross-encoder reranker was fine-tuned from `cross-encoder/ms-marco-MiniLM-L-12-v2` on Turkish legal relevance pairs and stored at `reranker_ft/`. The Qwen2.5-7B-Instruct model was fine-tuned using QLoRA (LoRA on the 4-bit quantized base model) and the adapter is stored at `qwen_qlora_ft/`. All fine-tuned models are hosted in the Kaggle dataset `ardayildiz29/legal-models-tr-finetuned` and loaded at runtime.

### 5. Evaluation Methodology

The Turkish legal RAG system was evaluated from multiple perspectives, including retrieval quality, answer quality, grounding, citation consistency, and hallucination behavior. Each evaluation example contains a legal question, a verified answer, and one or more gold source documents, allowing the system to be tested as a complete RAG pipeline.

Retrieval performance was measured using `Recall@5`, `Recall@10`, `MRR`, and `nDCG@10` to evaluate whether the correct legal sources were retrieved and ranked effectively. Answer quality was evaluated using `F1`, `ROUGE-L`, and semantic similarity to measure correctness and meaning preservation.

Grounding quality was measured with faithfulness and citation accuracy, which evaluate whether generated answers are supported by the retrieved legal context and correctly cited. In addition, hallucination analysis was performed to detect fabricated legal articles, unsupported claims, incorrect interpretations, and citation mismatches.

The final evaluation score combines retrieval (35%), answer quality (40%), and grounding (25%) to measure the overall reliability and effectiveness of the system.

### 6. Experiments and Ablation Study

#### 6.1 Stage Descriptions

**S1 – Baseline Dense RAG:** Uses base BGE-M3 embeddings with dense FAISS retrieval and base Qwen2.5-7B.

**S2 – Embedding FT:** Replaces the base embedding model with a fine-tuned Turkish legal embedding model.

**S3 – Sparse BM25:** Uses BM25-only retrieval as a lexical baseline.

**S4 – Hybrid Retrieval:** Combines dense retrieval and BM25 using Reciprocal Rank Fusion (RRF).

**S5 – Base Reranker:** Adds a base cross-encoder reranker on top of hybrid retrieval.

**S6 – FT Reranker:** Replaces the base reranker with a fine-tuned Turkish legal reranker.

**S7 – QLoRA LLM:** Uses a QLoRA fine-tuned Qwen2.5-7B model with hybrid retrieval.

**S8 – Full System:** Combines FT embeddings, hybrid retrieval, FT reranker, and QLoRA fine-tuned LLM.

## 6.2 Ablation Results Table

| Stage | R@5    | R@10   | MRR    | nDCG@5 | F1     | BLEU   | R-1    | R-L    | Faithfulness | Hallucination |
|-------|--------|--------|--------|--------|--------|--------|--------|--------|--------------|---------------|
| S1    | 0.7875 | 0.8250 | 0.6826 | 0.7044 | 0.3739 | 0.1104 | 0.3795 | 0.2903 | 0.6288       | 4             |
| S2    | 0.8792 | 0.9125 | 0.8248 | 0.8350 | 0.3584 | 0.1032 | 0.3675 | 0.2800 | 0.6250       | 7             |
| S3    | 0.7875 | 0.8250 | 0.6996 | 0.7157 | 0.3540 | 0.1043 | 0.3639 | 0.2775 | 0.5877       | 9             |
| S4    | 0.8708 | 0.9167 | 0.8181 | 0.8259 | 0.3559 | 0.0994 | 0.3680 | 0.2797 | 0.6197       | 4             |
| S5    | 0.8708 | 0.9167 | 0.8181 | 0.8259 | 0.3814 | 0.1363 | 0.4292 | 0.3376 | 0.5559       | 11            |
| S6    | 0.8708 | 0.9167 | 0.8181 | 0.8259 | 0.3818 | 0.1408 | 0.4308 | 0.3391 | 0.5803       | 10            |
| S7    | 0.8708 | 0.9167 | 0.8181 | 0.8259 | 0.5903 | 0.3402 | 0.5666 | 0.4813 | 0.8391       | 1             |
| S8    | 0.8708 | 0.9167 | 0.8181 | 0.8259 | 0.5958 | 0.3458 | 0.5630 | 0.4791 | 0.8443       | 1             |

## 7. Results

### 7.1 Base RAG vs Fully Optimized RAG

The comparison between Stage 1 (Baseline Dense RAG) and Stage 8 (Fully Optimized System) demonstrates the cumulative effect of all pipeline optimizations. Recall@5 improved from 0.7875 to 0.8708 (+10.6%), confirming that the optimized retrieval stack consistently surfaces the correct legal source within the top-5 results. MRR increased from 0.6826 to 0.8181, indicating that the relevant document is now ranked higher on average.

The most significant improvements are in answering quality metrics. F1 score increased from 0.3739 to 0.5958 (+59.3%), BLEU from 0.1104 to 0.3458 (+213.2%), and ROUGE-L from 0.2903 to 0.4791 (+65.0%). These gains are primarily attributable to the QLoRA fine-tuned language model, which produces more structured and citation-grounded answers.

Faithfulness improved from 0.6288 to 0.8443 (+34.3%), and hallucination suspects (faithfulness <

0.3) dropped from 4/240 to 1/240. This confirms that the fully optimized system generates answers that are substantially more grounded in the retrieved legal context compared to the baseline.

## 7.2 Component Contribution

| Metric                 | S1 Baseline | S8 Full System | Change  |
|------------------------|-------------|----------------|---------|
| Recall@5               | 0.7875      | 0.8708         | +0.0833 |
| Recall@10              | 0.825       | 0.9167         | +0.0917 |
| MRR                    | 0.6826      | 0.8181         | +0.1355 |
| nDCG@5                 | 0.7044      | 0.8259         | +0.1215 |
| F1                     | 0.3739      | 0.5958         | +0.2219 |
| BLEU                   | 0.1104      | 0.3458         | +0.2354 |
| ROUGE-1                | 0.3795      | 0.563          | +0.1835 |
| ROUGE-L                | 0.2903      | 0.4791         | +0.1888 |
| Faithfulness           | 0.6288      | 0.8443         | +0.2155 |
| Hallucination suspects | 4/240       | 1/240          | -3      |

## 8. Discussion

The experimental results show that each optimization component improved the Turkish legal RAG pipeline in a different way. Embedding fine-tuning improved semantic retrieval by better matching Turkish legal questions with relevant passages. Hybrid retrieval further improved performance by combining semantic search with BM25-based keyword matching, which is especially important for legal terms, law names, and article numbers.

Reranking improved the ordering of retrieved passages, allowing the LLM to receive more relevant legal context and generate more grounded answers. LLM fine-tuning and retrieval-aware prompting also improved answer quality, producing more structured and citation-consistent responses. However, the final answer quality still depends heavily on retrieval quality.

The main limitation of the project is that the benchmark does not fully cover all Turkish legal domains, and some source documents may contain noisy or outdated information. In addition, certain legal questions require multi-hop reasoning across multiple documents.

Future work may include expanding the benchmark with more expert-verified examples, improving multi-hop retrieval and citation verification, and optimizing inference latency for real-time use.

## 10. Conclusion

This project developed and evaluated a Turkish legal RAG system by comparing a standard Base RAG pipeline with a fully optimized RAG pipeline. The ablation study measured the effect of embedding fine-tuning, hybrid retrieval, reranking, and LLM fine-tuning under the same benchmark setting.

**Project Repository:** <https://github.com/29ardayildiz/turkish-legal-rag-pipeline>